Crowd Density Estimation using Haar Wavelet Transform and Support Vector Machines

Vibhu Nimalan Bharathi

June 9, 2025

1 Introduction

In crowded public spaces, effective crowd management is crucial to prevent dangerous situations from arising. When crowd density exceeds a critical threshold, the situation can quickly spiral out of control, leading to potential accidents, stampedes, or other hazards. Traditional pixel-based detection methods have been employed before to estimate crowd density, but these methods become unreliable as crowd density increases. At higher densities, pixel-based techniques often struggle due to improper edge detection and inadequate background removal, which severely impact the accuracy of density estimation.

To address these limitations, texture-based methods have emerged as more robust alternatives, offering better performance in challenging, high-density scenarios. Among these, the Haar wavelet transform has proven to be an effective tool for capturing the texture characteristics of crowded scenes. Coupled with Support Vector Machines (SVMs), this approach allows for a reliable estimation of crowd density by classifying it into four categories:

- **Low density**: 0 to 0.6 Ped/m^2
- **Moderate-low density**: 0.6 to 1.25 Ped/m^2
- **Moderate-high density**: 1.25 to 2 Ped/m^2
- **High density**: greater than 2 Ped/m^2

2 Crowd Density Examples

To illustrate the different density categories, Figure 1 shows representative images for each density class:

(a) Low density

(b) Moderate-low density

(c) Moderate-high density

(d) High density

Figure 1: Representative examples of different crowd density levels.

3 Algorithm

This paper will discuss a texture-based crowd density estimation approach developed by Li Xiaohua, Shen Lansun and Li Huanqin which consists of two main models:

1. **Density Character Extraction Model**: responsible for extracting relevant texture characteristics from the image using the Haar wavelet transform.

2. **Density Estimation Model**: uses the extracted features to estimate the crowd density using a Support Vector Machine classifier.

The subsequent sections will describe these models in detail, highlighting the feature extraction techniques and the classification strategies adopted.

3.1 Density Character Extraction Model

This model builds on the basic foundational idea that images with higher crowd density will have fine texture (higher frequency), and images with lower density will have coarse texture (lower frequency).

Haar wavelet transform helps bring out these texture details.

3.1.1 Pre-processing

The input image is first converted to grayscale, then histogram equalization is applied to normalize lighting variations, compensate for differences in camera gains, and improve overall contrast.

(a) Input (b) Grayscale (c) Equalised

Figure 2: Preprocessing

3.1.2 Wavelet Transform

The below equations are used to scale down the input images and produce 9 sub-images that enhance texture characteristics.

$$L_n(i,j) = [H_x * (H_y * L_{n-1}) \downarrow_{2,1}] \downarrow_{1,2} (i,j)$$

$$D_{n1}(i,j) = [H_x * (G_y * L_{n-1}) \downarrow_{2,1}] \downarrow_{1,2} (i,j)$$

$$D_{n2}(i,j) = [G_x * (H_y * L_{n-1}) \downarrow_{2,1}] \downarrow_{1,2} (i,j)$$

$$D_{n2}(i,j) = [G_x * (G_y * L_{n-1}) \downarrow_{2,1}] \downarrow_{1,2} (i,j)$$

$$H = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \end{bmatrix} \quad ; \quad G = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & -1 \end{bmatrix}$$

where:

- H: high-pass filter
- G: low-pass filter
- H_x/G_x: convolve along rows
- H_y/G_y: convolve along columns
- $*$: convolution operator

Hence nine sub-images are processed. $\{D_{nk}\}_{n=1,2,3;k=1,2,3}$

3.1.3 Obtained Sub-images

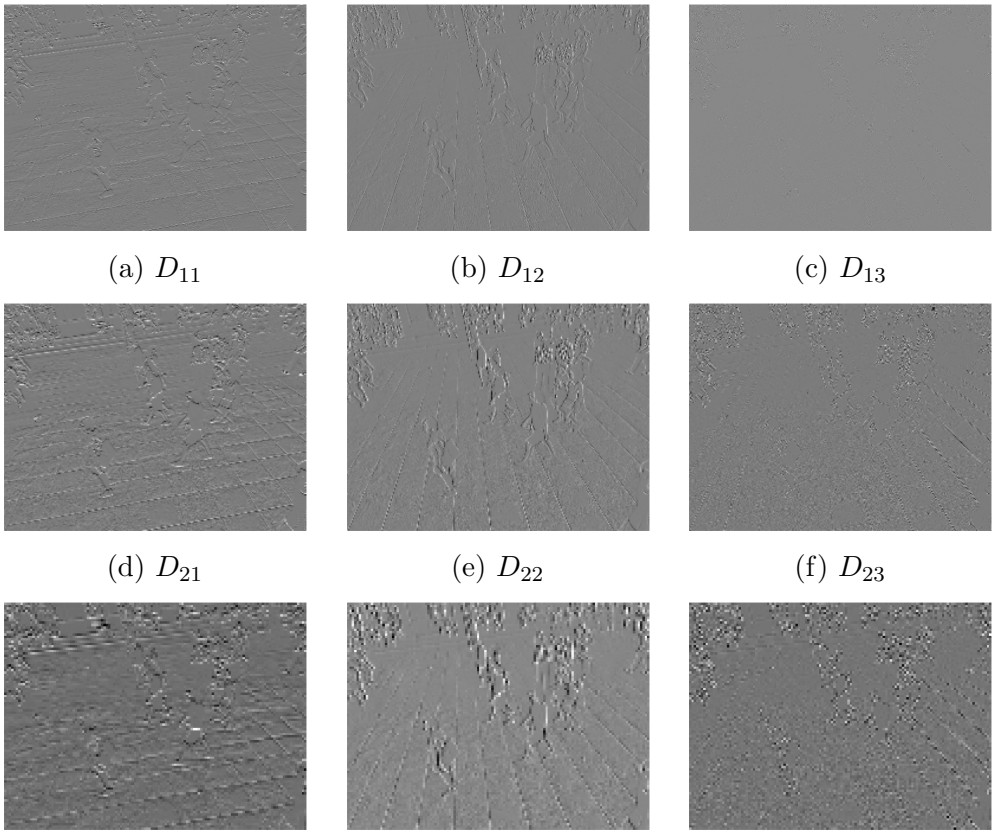
Figure 3 shows the obtained nine sub-images

3.1.4 Feature Extraction

1st order statistical features: We calculate normalized energy of sub images, they reflect distribution of energy along the frequency axis over different scales and directions and have proven to be powerful in texture characterization.

$$E_{nk} = \frac{1}{N} \sum_{i,j} \left(D_{nk}(i,j) \right).$$

The normalized energies $\{E_{nk}\}_{n=1,2,3;k=1,2,3}$ represent the first-order statistical features of a sub-image and are employed as a proximity indicator of density.



(a) D_{11} (b) D_{12} (c) D_{13}

(d) D_{21} (e) D_{22} (f) D_{23}

(g) D_{31} (h) D_{32} (i) D_{33}

Figure 3: Nine processed sub-images

2nd order statistical features We first quantize the sub-images so that we can create corresponding Grey Level Dependence matrix (GLDM):

$$D'_{nk} = \begin{cases} 0, & \text{if } D_{nk}^{\max} = D_{nk}^{\min} \\ \left\lfloor \frac{D_{nk} - D_{nk}^{\min}}{D_{nk}^{\max} - D_{nk}^{\min}} \cdot (I_{levels} - 1) \right\rfloor, & \text{otherwise} \end{cases}$$

where I_{levels} is the number of intensity levels (ranging from 0 to $I_{level} - 1$). Then we calculate the GLDM for each sub-image:

$$G_{n1}(l, m) = \frac{1}{M_n N_n} \sum_{i=1}^{M_n} \sum_{j=1}^{N_n} \begin{cases} 1, & D'_{n1}(i, j) = l, \quad D'_{n1}(i+1, j) = m \\ 0, & \text{otherwise} \end{cases}$$

$$G_{n2}(l,m) = \frac{1}{M_n N_n} \sum_{i=1}^{M_n} \sum_{j=1}^{N_n} \begin{cases} 1, & D'_{n2}(i,j) = l, \quad D'_{n2}(i,j+1) = m \\ 0, & \text{otherwise} \end{cases}$$

$$G_{n3}(l,m) = \frac{1}{M_n N_n} \sum_{i=1}^{M_n} \sum_{j=1}^{N_n} \begin{cases} 1, & D'_{n3}(i,j) = l, \quad D'_{n3}(i+1,j+1) = m \\ 0, & \text{otherwise} \end{cases}$$

where M_n,N_n denote the height and width of the sub-image D_{nk}.

Neighboring pixels in crowd images have a strong correlation with crowd density. GLDM are processed to capture this correlation.

Furthermore, GLDM obtained from fine texture are more uniformly distributed than a matrix obtained from images with coarse texture. Two indicators are calculated to measure the texture of images.

Homogeneity:

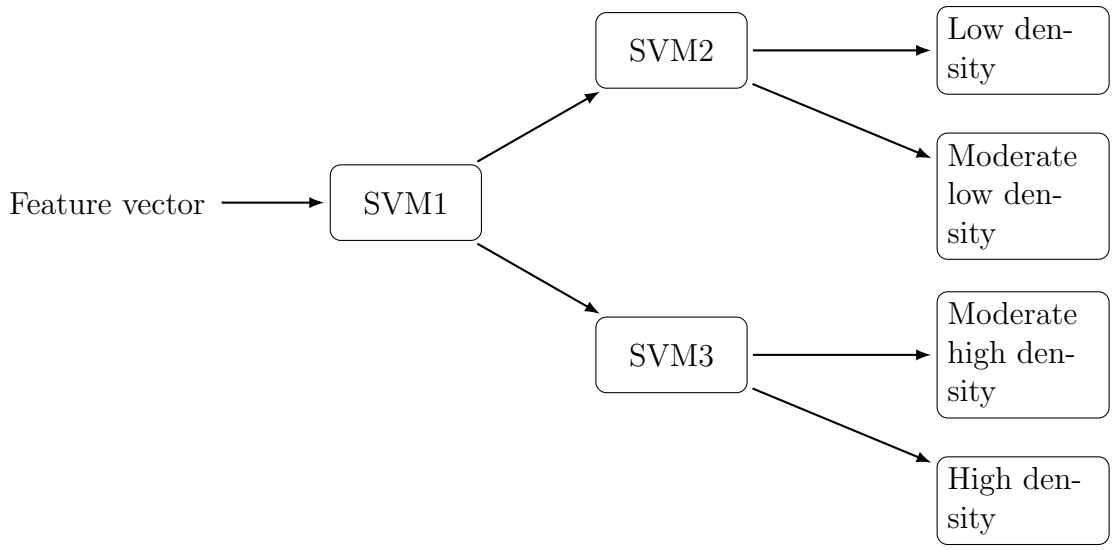
$$F^h_{nk} = \sum_{l,m=1}^{K} \frac{G_{nk}(l,m)}{1+(l-m)^2}$$

Contrast:

$$F^c_{nk} = \sum_{l,m=1}^{K} (l-m)^2 G_{nk}(l,m)$$

Hence with $\{E_{nk}, F^h_{nk}, F^c_{nk}\}_{n=1,2,3;k=1,2,3}$ we obtain our final 27-dimensional feature vector.

3.2 Density Estimation Model



Support Vector Machines (SVMs) are used to classify the input into the 4 categories. SVMs were originally proposed for categorization of two classes. Hence this uses three SVMs to categorize an input. SVM works by finding the best hyperplane that divides the data points into two distinct classes. SVM is formed by solving the problem of:

Maximize

$$Q(\alpha) = \sum_{i=1}^{N} \alpha_i - \frac{1}{2} \sum_{i=1}^{N} \sum_{j=1}^{N} \alpha_i \alpha_j y_i y_j K(x_i \cdot x_j)$$

Subject to

$$\sum_{i=1}^{N} y_i \alpha_i = 0, \quad 0 \leq \alpha_i \leq C, \quad i = 1, 2, \ldots, N.$$

where:

- y_i: class ($\{-1, 1\}$) of ith data point
- α_i: Lagrangian multiplier of ith data point
- $K(x_i.x_j)$: inner product kernel

After maximizing $Q(\alpha)$ we would classify the input x with:

$$f(x) = \text{sgn}\left(\sum_{x_i \in SV} \alpha_i y_i K(x_i, x) + b\right)$$

where:

- SV: set of support vectors

We can change the inner product kernel in cases where data points are not linearly seperable.

Table 1: Three typical inner-product functions and the type of decision surface they define

Inner-product function	**Type of classifier**	**Parameters**
$K(x_1, x_2) = [(x_1 \cdot x_2) + 1]^q$	Polynomial	q
$K(x_1, x_2) = \exp\left[-\frac{\lVert x_1 - x_2 \rVert^2}{\sigma^2}\right]$	Gaussian RBF	σ
$K(x_1, x_2) = \tanh[v(x_1 \cdot x_2) + c]$	Multi-layer perceptron	v, c

4 Results

4.1 Optimization

The Leave one out (LOO) method was used to choose the optimal inner product and the optimal hyperparameters for each SVM. A total of 300 data points were used for this model, 150 of these were used for training, and the rest were used for testing. For each inner-product function, a reasonable set of parameters is defined first. For instance the degree q/1,2,...,5 is often used for polynomial kernel. Then the optimal parameters for each SVM are selected by minimizing the LOO estimate of generalization error using a gradient descent algorithm over a set of parameters. Finally, the inner-product function, which has minimal estimate error, is selected. The resultant optimal inner-product functions and the corresponding parameters of the three SVMs for the training set of images are listed in Table 3.

Table 2: The inner-product function and hyperparameters of each SVM

	Inner-product function	Parameter	Percentage of SV
SVM1	Gaussian RBF	$\sigma = 0.6$	11.3%
SVM2	Gaussian RBF	$\sigma = 0.6$	10%
SVM3	Polynomial	$q = 4$	15%

4.2 Performance

The model achieves a Mean Average Error (MAE) of 0.0467% and a Mean Squared Error (MSE) of 0.0467%. For a 352×258 image, according to Li *et al* (2006) the model takes 100 milliseconds to classify the image. The wavelet transform, which generates 9 sub-images, consumes 60 milliseconds, while feature extraction (using GLDM, homogeneity, contrast, and energy) takes 20 milliseconds. The classification step itself also takes 20 milliseconds.

Table 3: Model results

	1 (40)	2 (40)	3 (30)	4 (40)	Test set correctness	Train. set correctness
1	40	0	0	0	95.3%	97.5%
2	0	37	1	0		
3	0	3	28	2		
4	0	0	1	38		
Error	0	3	2	2		

This model is a relatively light weight model. The overall complexity of this algorithm is $O(M.N)$ where M, N are the height and width of the input image. A modern computer can classify an input image of size $(352X258)$ image much faster than 100ms (≈10ms).

5 Conclusion

The proposed texture-based crowd density estimation approach, utilizing Haar wavelet transform and Support Vector Machines, offers a robust and efficient solution for classifying crowd density in public spaces. By leveraging the texture characteristics captured through wavelet decomposition and extracting both first-order and second-order statistical features, the method

effectively distinguishes between low, moderate-low, moderate-high, and high crowd density categories. The use of multiple SVMs with optimized inner-product kernels ensures high classification accuracy, achieving a test set correctness of 95.3% and a training set correctness of 97.5%, with a low Mean Average Error and Mean Squared Error of 0.0467%. Additionally, the algorithm's computational efficiency, with a processing time of approximately 100 milliseconds for a 352×258 image on older systems and potentially faster on modern hardware, makes it suitable for real-time applications. This approach outperforms traditional pixel-based methods, particularly in high-density scenarios, providing a reliable tool for crowd management and enhancing safety in crowded environments.